# CPU scheduling

- Preemptive
  - 1. Priority scheduling
  - 2. Shortest Remaining Job First
  - 3. Longest Remaining Job First
  - 4. Round Robin [RR]
- Non preemptive
  - 1. First come First serve [FCFS]
  - 2. Shortest Job First [SJF]
  - 3. Longest Job First
  - 4. Highest Response Ration next.

## (4) Priority scheduling [PS]

Priority scheduling is a CPU scheduling algorithm in which the process with the highest priority gets the CPU first.

> Important Rule :- Lower priority number = High Priority

Example:

| Priority number | meaning |
|---|---|
| 1 | Highest priority |
| 2 | Highest priority |
| 3 | medium priority |
| 4 | Low priority |

So, priority 1 > priority 2 > priority 3 > priority 4

Ex)

| Process | Arrival Time [AT] | Burst Time [BT] | Priority |
|---|---|---|---|
| P1 | 0 | 5 | 3 |
| P2 | 1 | 3 | 1 |
| P3 | 2 | 6 | 4 |
| P4 | 3 | 2 | 2 |

Smaller priority number = High priority

Step 1 :- If current time = 0

| Process | Priority |
|---|---|
| P1 | 3 |

only P1 is available.

so CPU executes P1

P1 : 0 → 5

current time becomes time = 5

Step 2 :- Now check which process have arrived before time 5.

| Process | Arrival Time |
|---|---|
| P2 | 1 |
| P3 | 2 |
| P4 | 3 |

All have arrived

Ready Queue

| Process | Priority |
|---|---|
| P2 | 1 |
| P3 | 4 |
| P4 | 2 |

Question : Which priority is highest ?

Priority 1

CPU select P2

P2 : 5 → 8

current time  time = 8

Step 3 Remaining process

| process | priority |
|---|---|
| P3 | 4 |
| P4 | 2 |

Highest Priority ? Priority 2

So CPU select

P4 : 8 → 10

current time  time = 10

Step 4 Remaining process

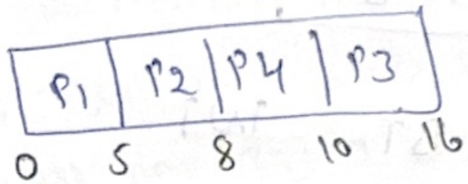
| process | priority |
|---|---|
| P3 | 4 |

only P3 is left

Execute it  P3 : 10 — 16

Now all process are completed

final Gantt chart




Completion Time [CT]

the Completion time is when each process finishes.

| process | CT |
|---|---|
| P1 | 5 |
| P2 | 8 |
| P4 | 10 |
| P3 | 16 |

Turnaround Time [TAT]

Formula : TAT = CT - AT

| process | CT | AT | TAT |
|---|---|---|---|
| P1 | 5 | 0 | 5 - 0 = 5 |
| P2 | 8 | 1 | 8 - 1 = 7 |
| P3 | 16 | 2 | 16 - 2 = 14 |
| P4 | 10 | 3 | 10 - 3 = 7 |

## Waiting time [WT]

**Formula: WT = TAT - BT**

| Process | TAT | BT | WT |
|---|---|---|---|
| P1 | 5 | 5 | 5-5 = 0 |
| P2 | 7 | 3 | 7-3 = 4 |
| P3 | 14 | 6 | 14-6 = 8 |
| P4 | 7 | 2 | 7-2 = 5 |

## Average Waiting time

$$= \frac{0+4+8+5}{4} = \frac{17}{4} = 4.25 \text{ ms}$$

## Average Turnaround time

$$= \frac{5+7+14+7}{4} = 33/4 = 8.25 \text{ ms}$$

## (7) Shortest Remaining Job First (SRJF)

SRJF is the preemptive version of shortest job first (SJF) scheduling. The CPU always executes the process with the smallest remaining burst time. If new process arrives with a shorter remaining time than the currently running process, the CPU immediately switches to the new process.

Example

| Process | Arrival time (AT) | Burst time (BT) |
|---|---|---|
| P1 | 0 | 8 |
| P2 | 1 | 4 |
| P3 | 2 | 2 |
| P4 | 3 | 1 |

Step 1: time = 0

Only P1 has arrived

| Process | Remaining time |
|---|---|
| P1 | 8 |

CPU executes P1. 0 ------ 1 P1

Remaining time of P1 = 7

Step 2: time = 1

Process P2 arrive

| Process | Remaining time |
|---|---|
| P1 | 7 |
| P2 | 4 |

since 4 < 7; P2 is shorter.

CPU switches from P1 to P2 0---1---2
P1 P2

Remaining time

P2 = 3
P1 = 7

Step 3: time = 2

Process P3 arrive

| Process | Remaining time |
|---|---|
| P1 | 7 |
| P2 | 3 |
| P3 | 2 |

Shortest remaining time = P3

CPU switches to P3 0---1---2---3
P1 P2 P3

Remaining

P3 = 1

Step 4) time = 3 process P4 arrive

| Process | Remaining time |
|---|---|
| P1 | 7 |
| P2 | 3 |
| P3 | 1 |
| P4 | 1 |

Both P3 and P4 have the same remaining time (1ms).

we FCFS to break the tie, so P3 continues because it arrived earlier.

0---1---2---3---4
P1 P2 P3 P3

Step 5: time = 4

Ready Queue

| Process | Remaining time |
|---|---|
| P1 | 7 |
| P2 | 3 |
| P4 | 1 |

Shortest = P4

CPU executes P4 0---1---2---3---4---5
P1 P2 P3 P3 P4

P4 finishes

Step 6: Time = 5

Ready Queue

| Process | Remaining time |
|---|---|
| P1 | 7 |
| P2 | 3 |

shortest = P2

CPU executes P2

0---1---2---3---4---5---6---7---8
P1 P2 P3 P3 P4 P2 P2 P2

P2 finishes at time = 8

step 7! time = 8
only P1 remains
Remaining time = 7
CPU executes P1 until completion.

8---9---10---11---12---13---14---15
P1 P1 P1 P1 P1 P1 P1

P1 finishes at time = 15

final Gantt chart

| P1 | P2 | P3 | P4 | P2 | P1 |
|---|---|---|---|---|---|
| 0 | 1 | 2 | 4 | 5 | 8 | 15

step 8! Completion time (CT)

| Process | CT |
|---|---|
| P1 | 15 |
| P2 | 8 |
| P3 | 4 |
| P4 | 5 |

step 9! Turnaround time (TAT)

formula! TAT = CT - AT

| Process | CT | AT | TAT |
|---|---|---|---|
| P1 | 15 | 0 | 15-0=15 |
| P2 | 8 | 1 | 8-1=7 |
| P3 | 4 | 2 | 4-2=2 |
| P4 | 5 | 3 | 5-3=2 |

step 10) Waiting time (WT)

formula! WT = TAT - BT

| Process | TAT | BT | WT |
|---|---|---|---|
| P1 | 15 | 8 | 15-8=7 |
| P2 | 7 | 4 | 7-4=3 |
| P3 | 2 | 2 | 2-2=0 |
| P4 | 2 | 1 | 2-1=1 |

step 11! Average waiting time

$$= \frac{7+3+0+1}{4} = \frac{11}{4} = 2.75 \text{ ms}$$

step 12! Average turnaround time

$$= \frac{15+7+2+2}{4} = \frac{26}{4} = 6.5 \text{ ms}$$

final Result

| Process | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| P1 | 0 | 8 | 15 | 15 | 7 |
| P2 | 1 | 4 | 8 | 7 | 3 |
| P3 | 2 | 2 | 4 | 2 | 0 |
| P4 | 3 | 1 | 5 | 2 | 1 |

## (8) Longest Remaining Job First (LRJF)

LRJF is the preemtive version of longest job first scheduling.

- the CPU always selects the process with the largest remaining burst time.
- If new process arrives with a longer remaining burst time than currently running process, the current process is preempted (stopped) and the new process gets the CPU.

Rule: Choose the process with the highest remaining CPU burst time.

Example

| Process | Arrival Time (AT) | Burst Time (BT) |
|---|---|---|
| P1 | 0 | 4 |
| P2 | 1 | 6 |
| P3 | 2 | 2 |

Step 1: Time = 0

Only P1 has arrived

| Process | Remaining Time |
|---|---|
| P1 | 4 |

CPU executes P1

Time : 0 —P1→ 1

After 1ms:

P1 Remaining = 4 − 1 = 3

Step 3: Time = 1 now P2 arrives

| Process | RT |
|---|---|
| P1 | 3 |
| P2 | 6 ← largest remaining time |

So P2 gets the CPU

Time : 0 —P1→ 1 —P2→ 2

after 1ms;

P2 Remaining = 6 − 1 = 5

Step 4: Time = 2 now P3 arrives

| Process | RT |
|---|---|
| P1 | 3 |
| P2 | 5 ← largest remaining time |
| P3 | 2 |

So P2 continues

Time : 0 —P1→ 1 —P2→ 2 —P2→ 3

Remaining : P2 = 5 − 1 = 4

Step 5: Time = 3

| Process | RT |
|---|---|
| P1 | 3 |
| P2 | 4 ← largest remaining time |
| P3 | 2 |

So P2 continues

Time : 0 — 1(P1) — 2(P2) — 3(P2) — 4(P2)

Remaining : P2 = 4-1 = 3

Step 6) time = 4

| Process | RT |
|---|---|
| P1 | 3 |
| P2 | 3 |
| P3 | 2 |

P1 and P2 both have 3rd remaining

Tie rule : Continue the currently running process

So P2 continues

Time : 0 — 1(P1) — 2(P2) — 3(P2) — 4(P2) — 5(P2)

Remaining ; P2 = 3-1 = 2

Step 7) time = 5

| Process | RT |
|---|---|
| P1 | 3 ← largest remaining time |
| P2 | 2 |
| P3 | 2 |

So the CPU switches from P2 to P1

Time : 0 — 1(P1) — 2(P2) — 3(P2) — 4(P2) — 5(P2) — 6(P1)

Remaining : P1 = 3-1 = 2

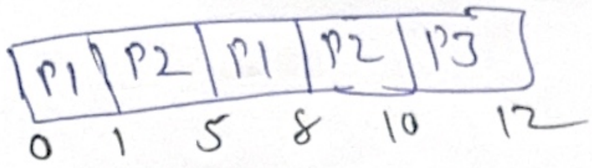
Step 8) Remaining Execution

now all process have 2nd remaining

we continue with the current process (P1)

- P1 runs from 6 to 8 → finished
- P2 runs from 8 to 10 → finished
- P3 runs from 10 to 12 → finished

Final Gantt chart




Completion Time (CT)

| Process | CT |
|---|---|
| P1 | 8 |
| P2 | 10 |
| P3 | 12 |

## (3) Round Robin scheduling [RR]

Round Robin is a preemptive CPU scheduling algorithm in which each process gets a fixed amount of CPU time called a time quantum (time slice).

* Each process is executed for one time quantum.
* If the process finishes within the time quantum, it leaves the queue.
* If it does not finish, it is moved to the end of the ready queue.
* The CPU continues executing the next process in the queue.
* The continues until all processes are completed.

### Diagram

Ready queue

Front → P1 | P2 | P3 | P4 ← Rear

CPU executes one process for time quantum

If completed → Exit

If not completed → Move to Rear

Example

| Process | Burst Time [BT] |
|---|---|
| P1 | 10ms |
| P2 | 5 |
| P3 | 8 |
| P4 | 6 |

Time Quantum = 3ms

Step 1 : Ready Queue $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$

Execute P1

Burst Time = 10

Time Quantum = 3

CPU executes only 3ms

Remaining Burst Time : 10 - 3 = 7ms

| Process | Remaining |
|---|---|
| P1 | 7 |

0 — 3
P1

Step 2 : Execute P2

Burst Time = 5ms

CPU executes 3ms

Remaining Burst Time : 5-3 = 2ms

| Process | Remaining |
|---|---|
| P2 | 2 |

0 — 3 — 6
P1 P2

Step 3 : Execute P3

BT = 8ms

CPU executes 3ms

Remaining : 8 - 3 = 5ms

0 — 3 — 6 — 9
P1 P2 P3

Step 4 : Execute P4

BT = 6ms

CPU executes 3ms

Remaining : 6 - 3 = 3ms

0 --- 3 -- 6 --- 9 -- 12
P1 P2 P3 P4

Current Remaining times

| Process | Remaining |
|---|---|
| P1 | 7 |
| P2 | 2 |
| P3 | 5 |
| P4 | 3 |

Step 5 : Execute P1 again

Remaining = 7ms

CPU executes 3ms

Remaining : 7 - 3 = 4ms

| Process | Remaining |
|---|---|
| P1 | 4 |

0 --- 3 --- 6 --- 9 --- 12 --- 15
P1 P2 P3 P4 P1

Step 6 : Execute P2

Remaining BT = 2ms

Time Quantum = 3ms

Can the CPU execute 3ms?

Because P2 has only 2ms of work left.

so it executes only 2ms and finishes.

Remaining = 2ms

Execute = 2ms

Remaining : 2-2 = 0

P2 finishes

0---3---6---9---12---15---17
P1 P2 P3 P4 P1 P2

**Step 7** Execute P3

Remaining BT = 5ms

Execute = 3ms

Remaining : 5-3 = 2ms

0---3---6---9---12---15---17---20
P1 P2 P3 P4 P1 P2 P3

**Step 8:** Execute P4

Remaining BT = 3ms

Execute = 3ms

Remaining : 3-3 = 0

P4 finishes

0---3---6---9---12---15---17---20---23
P1 P2 P3 P4 P1 P2 P3 P4

**Step 9:** Execute P1

Remaining BT = 4ms

Execute = 3ms

Remaining : 4-3 = 1ms

0---3---6---9---12---15---17---20---23---26
P1 P2 P3 P4 P1 P2 P3 P4 P1

**Step 10:** Execute P3

Remaining = 2ms

Time quantum = 3ms

Can CPU execute 3ms?

only 2ms is left

Remaining : 2-2 = 0

so P3 finishes

0---3---6---9---12---15---17---20---23---26---28
P1 P2 P3 P4 P1 P2 P3 P4 P1 P3

**Step 11:** Execute P1

Remaining = 1ms

Time quantum = 3ms

Can CPU execute 3ms?

only 1ms remain

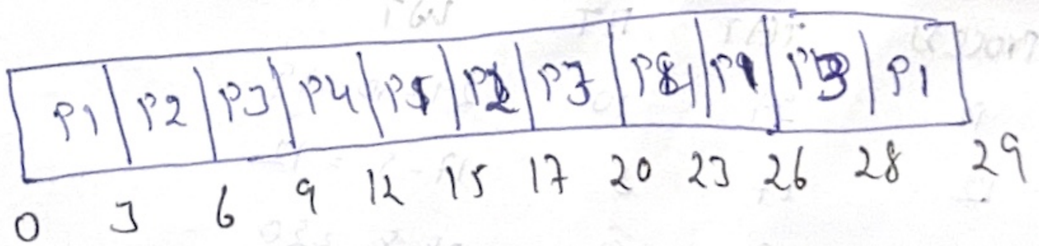
Remaining : 1-1 = 0

P1 finishes

0---3---6---9---12---15---17---20---23---26---28---29
P1 P2 P3 P4 P1 P2 P3 P4 P1 P3 P1

all process are finished.

Gantt chart



## Completion Time (CT)

| Process | Completion Time |
|---|---|
| P1 | 29 |
| P2 | 17 |
| P3 | 28 |
| P4 | 23 |

## Turnaround Time [TAT]

Formula : TAT = Completion Time - Arrival Time

All process arrive at time 0:

| Process | CT | Arrival Time | TAT |
|---|---|---|---|
| P1 | 29 | 0 | 29-0 = 29 |
| P2 | 17 | 0 | 17-0 = 17 |
| P3 | 28 | 0 | 28-0 = 28 |
| P4 | 23 | 0 | 23-0 = 23 |

## Waiting Time [WT]

Formula : WT = Turnaround Time - Burst Time

| Process | TAT | BT | WT |
|---|---|---|---|
| P1 | 29 | 10 | 29-10 = 19 |
| P2 | 17 | 5 | 17-5 = 12 |
| P3 | 28 | 8 | 28-8 = 20 |
| P4 | 23 | 6 | 23-6 = 17 |

Average Waiting Time

$$= \frac{19+12+20+27}{4} = \frac{68}{4} = 17 \text{ ms}$$

Average Turnaround Time

$$= \frac{29+17+28+23}{4} = \frac{97}{4} = 24.25 \text{ ms}$$

## Advantage

- Fair CPU allocation
- Every process get CPU time
- No starvation
- Better response time
- Easy to implement.

## Disadvantage

- Too small time quantum → Many context switches
- Too large time quantum → Become similar to FCFS
- Higher context switch overhead.

## (2) First Come First Serve (FCFS) Scheduling

- FCFS is the simplest CPU scheduling in an OS

In FCFS scheduling the process that arrives first gets the CPU first. It follows the FIFO (First in, first out) principle, just like people standing in a queue at a ticket counter.

- first process to enter the ready queue → Executed first.
- non-preemptive scheduling algorithm (once a process starts executing, it cannot be interrupted until it finishes).

Real-life Example

Imagine there are four students waiting to submit an assignment

| arrival order | student |
|---|---|
| 1st | A |
| 2nd | B |
| 3rd | C |
| 4th | D |

The teacher checks them in the same order.

A → B → C → D

FCFS scheduling works exactly like this.

Example:

| Process | Arrival Time (AT) | Burst Time (BT) |
|---|---|---|
| P1 | 0 | 5 |
| P2 | 1 | 3 |
| P3 | 2 | 8 |
| P4 | 3 | 6 |

Step 1: Arrange by AT

The order is already correct P1 → P2 → P3 → P4

Step 2: Draw the Gantt chart

P1 Arrived at time = 0

CPU is free

Run for 5 ms

Start = 0

End = 5

0 ---- 5
P1

P2 P2 arrived at time 1 but wait until P1 finishes.

Start = 5

Runs for 3 ms

End = 8

5 ---- 8
P2

P3 P3 arrived at time 2 but wait until P2 finishes.

Start = 8

Runs for 8 ms

End = 16

8 ---- 16
P3

## (2) First Come First serve (FCFS) Scheduling

* FCFS is the simplest CPU scheduling in an OS.

In FCFS scheduling the process that arrives first gets the CPU first. It follows the FIFO (First in, First out) principle, just like people standing in a queue at a ticket counter.

* first process to enter the ready queue → Executed first.
* non-preemptive scheduling algorithm (once a process starts executing, it cannot be interrupted until it finishes).

Real-life Example

Imagine there are four students waiting to submit an assignment

| Arrival order | Student |
|---|---|
| 1st | A |
| 2nd | B |
| 3rd | C |
| 4th | D |

The teacher checks them in the same order.

A → B → C → D

FCFS scheduling works exactly like this.

Example:

| Process | Arrival Time (AT) | Burst Time (BT) |
|---|---|---|
| P1 | 0 | 5 |
| P2 | 1 | 3 |
| P3 | 2 | 8 |
| P4 | 3 | 6 |

Step 1: Arrange by AT

The order is already correct P1 → P2 → P3 → P4

Step 2: Draw the Gantt chart

P1 Arrived at time = 0

CPU is free

Run for 5 ms

Start = 0

End = 5

0 ---- 5
P1

P2 P2 arrived at time 1 but wait until P1 finishes.

Start = 5

Runs for 3 ms

End = 8

5 ---- 8
P2

P3 P3 arrived at time 2 but wait until P2 finishes.

Start = 8

Runs for 8 ms

End = 16

8 ---- 16
P3

P4 starts after P3.

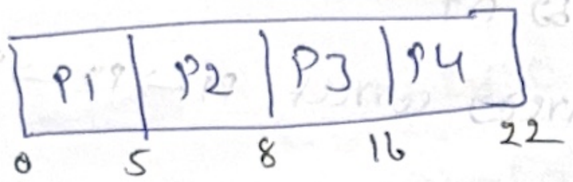
Starts = 16

Runs for 6ms

Ends = 22

16 ---- 22
P4

Final Gantt chart




Step-3: Completion Time (CT)

Completion Time = Time at which the process finishes.

| Process | CT |
|---|---|
| P1 | 5 |
| P2 | 8 |
| P3 | 16 |
| P4 | 22 |

Step-4:- Turnaround Time [TAT]

Formula TAT = CT - AT

| Process | CT | AT | TAT |
|---|---|---|---|
| P1 | 5 | 0 | 5-0 = 5 |
| P2 | 8 | 1 | 8-1 = 7 |
| P3 | 16 | 2 | 16-2 = 14 |
| P4 | 22 | 3 | 23-3 = 19 |

Step 5: Waiting Time (WT)

Formula : WT = TAT - BT

| Process | TAT | BT | WT |
|---|---|---|---|
| P1 | 5 | 5 | 5-5 = 0 |
| P2 | 7 | 3 | 7-3 = 4 |
| P3 | 14 | 8 | 14-8 = 6 |
| P4 | 19 | 6 | 19-6 = 13 |

Step 6: Average Waiting Time

Formula Average WT = $\frac{WT1+WT2+WT3+WT4}{\text{Number of Processes}}$

$= \frac{0+4+6+13}{4} = \frac{23}{4} = 5.75$ ms

Step 7: Average Turnaround Time

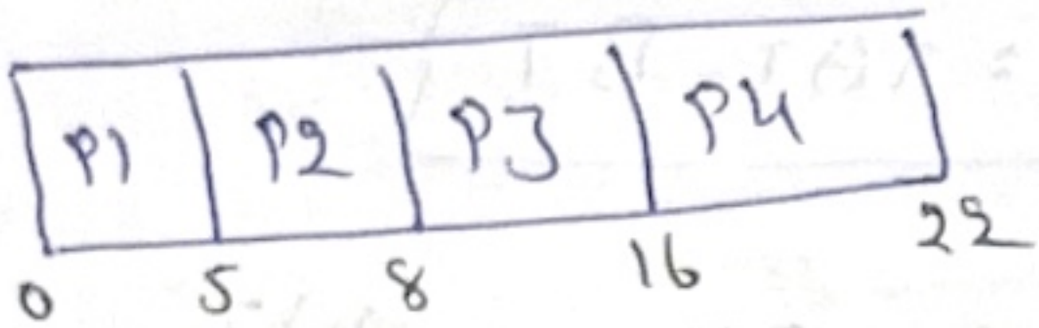
Formula Average TAT = $\frac{5+7+14+19}{4}$

$= \frac{45}{4} \Rightarrow 11.25$ ms

Final Table

| Process | AT | BT | CT | TAT=CT-AT | WT = TAT-BT |
|---|---|---|---|---|---|
| P1 | 0 | 5 | 5 | 5 | 0 |
| P2 | 1 | 3 | 8 | 7 | 4 |
| P3 | 2 | 8 | 16 | 14 | 6 |
| P4 | 3 | 6 | 22 | 19 | 13 |

Gantt chart




Average waiting time (AWT) = 5.75 ms

Average turnaround time (ATAT) = 11.25 ms

Advantages

- very simple and easy to implement
- fair because processes are executed in arrival order
- No starvation [every process eventually gets CPU time]

Disadvantages

- Poor average waiting time when a long job arrive first
- not suitable for interactive or time sharing system.

## ① Shortest Job First (SJF) Scheduling.

Shortest Job first is a CPU scheduling algorithm in which the process with the smallest CPU burst time is executed first

* CPU burst time = time required by a process to complete its execution

* SJF is mainly used to minimize the average waiting time

* It is one of the most efficient scheduling algorithms when the CPU burst times are known.

millisecond [ms]

### Key features

- non-preemptive scheduling
- Executes the shortest job first
- Reduces average waiting time
- Better CPU utilization.

### Example

| Process | Arrival Time (AT) | Burst Time (BT) |
|---|---|---|
| P1 | 0 | 6 |
| P2 | 1 | 8 |
| P3 | 2 | 7 |
| P4 | 3 | 3 |

Time : 0, 1 2 3 4 5 .....

**Step 1:** Time = 0

only P1 has arrived.

Ready queue

| Process | BT |
|---|---|
| P1 | 6 |

Execute P1 0 ---- 6 ($P_1$)

Completed P1

**Step 2:** Time = 6

now all remaining process have arrived

Ready queue

| Process | BT |
|---|---|
| P2 | 8 |
| P3 | 7 |
| P4 | 3 |

Choose the smallest burst ...

smallest = P4 (3ms)

Execute P4 6 ---- 9
P4

Completed P1, P4

Step-3 Time = 9

Remaining process

| process | BT |
|---|---|
| P2 | 8 |
| P3 | 7 |

smallest Burst time

Execute P3 9 ---- 16
P3

Completed P1 P4 P3

Step 4: Time = 16

only one process remains

| process | BT |
|---|---|
| P2 | 8 |

Execute P2 16 ---- 24
P2

All process Completed.

Final Gantt chart — | P1 | P4 | P3 | P2 |
0 6 9 16 24

Execution order — P1 → P4 → P3 → P2

Step-5 Calculate Completion time (CT)

Completion time = Time at which the process finishes.

| process | CT |
|---|---|
| P1 | 6 |
| P4 | 9 |
| P3 | 16 |
| P2 | 24 |

Step-6 Calculate Turnaround time (TAT)

formula : TAT = CT - AT

| process | CT | AT | TAT |
|---|---|---|---|
| P1 | 6 | 0 | 6 |
| P2 | 24 | 1 | 23 |
| P3 | 16 | 2 | 14 |
| P4 | 9 | 3 | 6 |

Step 7: Calculate waiting time (WT)

Formula : WT = TAT - BT

| Process | TAT | BT | WT |
|---|---|---|---|
| P1 | 6 | 6 | 0 |
| P2 | 23 | 8 | 15 |
| P3 | 14 | 7 | 7 |
| P4 | 6 | 3 | 3 |

Step 8: Average waiting time = Sum of all values / Number of values

Average WT = $\frac{0+15+7+3}{4}$ = 25/4

= 6.25 ms

Step 9: Average Turnaround time

Average TAT = $\frac{6+23+14+6}{4} = \frac{49}{4}$ = 12.25 ms

Final table

| Process | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| P1 | 0 | 6 | 6 | 6 | 0 |
| P2 | 1 | 8 | 24 | 23 | 15 |
| P3 | 2 | 7 | 16 | 14 | 7 |
| P4 | 3 | 3 | 9 | 6 | 3 |

Advantage:

- minimizes average waiting time
- Efficient for short jobs
- Improves system throughput.
- Better CPU utilization

Disadvantage

- Burst time must be know in advance
- Long process may experience starvation if short jobs keep arriving.
- not suitable for interactive or real-time systems

## ⑤ Longest Job First [LJF]

Longest Job First [LJF] is a non-preemptive CPU scheduling algorithm in which the process with the largest Burst time [BT] is executed first.

Longest = Highest Burst time

Job = process

First = selected before shorter job

non-preemptive = once a process starts executing, it runs until it finishes.

Example: Suppose all processes arrive at time = 0

| Process | Arrival Time [AT] | Burst Time [BT] |
|---|---|---|
| P1 | 0 | 5ms |
| P2 | 0 | 12ms |
| P3 | 0 | 3ms |
| P4 | 0 | 8ms |

STEP 1: All processes arrive

At Time = 0 - Ready Queue

| Process | Burst Time |
|---|---|
| P1 | 5 |
| P2 | 12 |
| P3 | 3 |
| P4 | 8 |

Now compare burst time

$P_1 = 5$

$P_2 = 12$ ← longest

$P_3 = 3$

$P_4 = 8$

select P2 because it has the largest burst time

Step 2: Execute P2

Time 0 ------ 12

P2

P2 finishes at 12ms

Remaining processes.

| Process | BT |
|---|---|
| P1 | 5 |
| P3 | 3 |
| P4 | 8 |

Step 3: Choose the longest Remaining Job

$P_1 = 5$

$P_3 = 3$

$P_4 = 8$ ← longest

Execute P4

12 ------ 20

P4

P4 finisher at 20ms

Remaining: Process BT

| | |
|---|---|
| P1 | 5 |
| P3 | 3 |

Step 4: Choose longest

$P_1 = 5$ ← longest

$P_3 = 3$

Execute P1: 20 ------ 25

P1

P1 finisher at 25ms

Remaining: Process BT

| | |
|---|---|
| P3 | 3 |

Step 5: Execute the last process

Execute P3.

25 ----- 28

P3

P3 finishers at 28ms

Final Gantt chart

| 0 | 12 | 20 | 25 | 28 |
|---|---|---|---|---|
| P2 | P4 | P1 | P3 | |

Execution order: P2 → P4 → P1 → P3

Step 6: Completion Time (CT)

| Process | CT |
|---|---|
| P2 | 12 |
| P4 | 20 |
| P1 | 25 |
| P3 | 28 |

Step 7: Turnaround time [TAT]

formula: TAT = CT - AT

| Process | CT | AT | TAT |
|---|---|---|---|
| P1 | 25 | 0 | 25-0 = 25 |
| P2 | 12 | 0 | 12-0 = 12 |
| P3 | 28 | 0 | 28-0 = 28 |
| P4 | 20 | 0 | 20-0 = 20 |

Step 8: Waiting time [WT]

formula: WT = TAT - BT

| Process | TAT | BT | WT |
|---|---|---|---|
| P1 | 25 | 5 | 25-5 = 20 |
| P2 | 12 | 12 | 12-12 = 0 |
| P3 | 28 | 3 | 28-3 = 25 |
| P4 | 20 | 8 | 20-8 = 12 |

Step 9: Average waiting time

$$AWT = \frac{20+0+25+12}{4} = \frac{57}{4} = 14.25 \text{ ms}$$

AWT = 14.25 ms

Step 10: Average Turnaround time

$$ATT = \frac{25+12+28+20}{4} = \frac{85}{4} = 21.25 \text{ ms}$$

Average Turnaround Time = 21.25 ms

Final table

| Process | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| P1 | 0 | 5 | 25 | 25 | 20 |
| P2 | 0 | 12 | 12 | 12 | 0 |
| P3 | 0 | 3 | 28 | 28 | 25 |
| P4 | 0 | 8 | 20 | 20 | 12 |

Advantages

- Simple to understand and implement.
- long jobs finish early.
- suitable when long tasks are more important than short tasks.

Disadvantages

- short processes may wait for a very long time
- Poor average waiting time for short jobs.
- Not suitable for interactive or real-time systems.

## ⑥ Highest Response Ration Next

HRRN is a non-preemptive CPU scheduling algorithm

- It selects the process with the highest Response Ratio(RR)
- HRRN is an improvement over SJF because it reduces the chance of starvation.

Response Ratio Formula [RR] = $\frac{\text{Waiting time} + \text{Burst Time}}{\text{Burst Time}}$

OR

$$RR = 1 + \frac{\text{Waiting Time}}{\text{Burst Time}}$$

Where: WT = Waiting Time

BT = Burst time

The process with the highest RR is selected.

Example

| Process | Arrival Time[AT] | Burst Time[BT] |
|---|---|---|
| P1 | 0 | 8ms |
| P2 | 1 | 4ms |
| P3 | 2 | 9ms |
| P4 | 3 | 5ms |

Step 1: Time = 0

Only P1 has arrived

| Ready Queue | Process | AT | BT |
|---|---|---|---|
| | P1 | 0 | 8 |

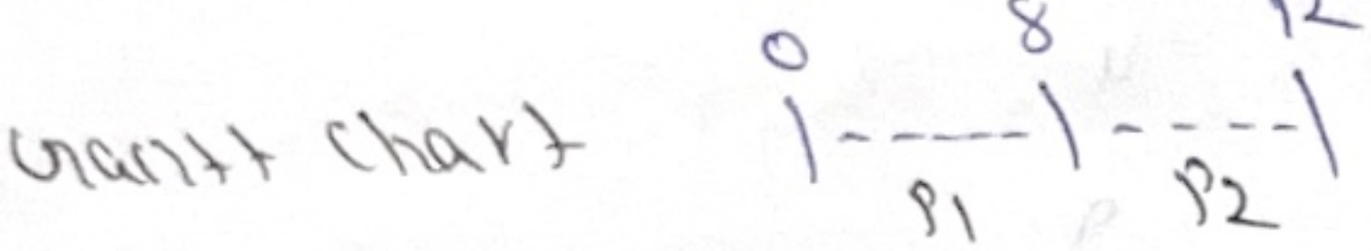
Compare Response Ratio of

| Process | RR |
|---|---|
| P2 | 2.75 |
| P3 | 1.67 |
| P4 | 2.00 |

Highest RR = P2

Execute P2

Gantt chart




P2 finishes at 12 ms

At P3: Time = 12

Remaining:

| Process | AT | BT |
|---|---|---|
| P3 | 2 | 9 |
| P4 | 3 | 5 |

Process P3

Waiting time = 12 − 2 = 10

$$RR = \frac{10+9}{9} = \frac{19}{9} = 2.11$$

Process P4

$$WT = 12 - 3 = 9$$

$$RR = \frac{9+5}{5}$$

$$= \frac{14}{5}$$

$$= 2.8$$

Step 6:- Turnaround time (TAT)

formula: TAT = CT - AT

| Process | CT | AT | TAT |
|---|---|---|---|
| P1 | 8 | 0 | 8-0 = 8 |
| P2 | 12 | 1 | 12-1 = 11 |
| P3 | 26 | 2 | 26-2 = 24 |
| P4 | 17 | 3 | 17-3 = 14 |

Step 7: Waiting time (WT)

formula: WT = TAT - BT

| Process | TAT | BT | WT |
|---|---|---|---|
| P1 | 8 | 8 | 8-8 = 0 |
| P2 | 11 | 4 | 11-4 = 7 |
| P3 | 24 | 9 | 24-9 = 15 |
| P4 | 14 | 5 | 14-5 = 9 |

Step 8: Average waiting time

$$AWT = \frac{0+7+15+9}{4} = \frac{31}{4} = 7.75 \text{ ms}$$

Step 9: Average Turnaround time

$$ATT = \frac{8+11+24+14}{4} = \frac{57}{4} = 14.25 \text{ ms}$$

Final Result

| Process | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| P1 | 0 | 8 | 8 | 8 | 0 |
| P2 | 1 | 4 | 12 | 11 | 7 |
| P3 | 2 | 9 | 26 | 24 | 15 |
| P4 | 3 | 5 | 17 | 14 | 9 |